

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №4
по курсу «Параллельная обработка данных»»**

Сортировка чисел на GPU. Свертка, сканирование, гистограмма.

Выполнил: И.А. Хомяков
Группа: 8О-407Б
Преподаватель: А.Ю. Морозов

Москва, 2024

Условие

1. Цель работы: Ознакомление с фундаментальными алгоритмами GPU: свертка (reduce), сканирование (blelloch scan) и гистограмма (histogram). Реализация одной из сортировок на CUDA. Использование разделяемой и других видов памяти. Исследование производительности программы с помощью утилиты nvprof (обязательно отразить в отчете).
2. Вариант 8: Поразрядная сортировка.

Программное и аппаратное обеспечение

Compute capability: 7.5

Name: Tesla T4

Total Global Memory: 15835660288

Shared memory per block: 49152

Registers per block: 65536

Warp size: 32

Max threads per block: (1024, 1024, 64)

Max block: (2147483647, 65535, 65535)

Total constant memory: 65536

Multiprocessors count: 40

Метод решения

Поразрядная сортировка, в которой используется blelloch scan для сортировки битов всех чисел. Алгоритм blelloch можно удобно распараллелить как горизонтально, распараллеливая вычисление сумм, так и вертикально, так как нам не обязательно дожидаться вычисления текущего уровня, чтобы перейти на следующий в текущем потоке. В своей программе реализовал рекурсивный подход, который параллелит только горизонтальные вычисления.

Важна устойчивость при сортировке по разрядам. После префиксного суммирования мы расставляем числа по своим местам для текущего разряда. Если бит числа в текущем разряде 0, то мы его стараемся переместить в начало: текущий индекс минус количество единичным битов для этого индекса. Если бит не нулевой, то вычисляем позицию от конца списка: к количеству элементов прибавляем количество единичных битов на текущем индексе и вычитаем общее количество единичных битов.

Устойчивость обеспечивается за счет того, что мы до перестановки вычислили префиксную сумму и используем ее для вычисления новых позиций чисел.

Для вычисления префиксной суммы используется функция upDownSweep. Сам blelloch scan разбивается на up-sweep и down-sweep. На первом этапе строим дерево сумм, где каждый узел - сумма дочерних узлов. Начинаем с этапа, где все элементы массива - листья. На каждом этапе объединяем попарно элементы, получается дерево.

Количество листьев должно быть степенью двойки для удобства. На этапе down-sweep мы вычисляем префиксные суммы для каждого элемента начиная с корня дерева,

которое мы устанавливаем в 0. Проходим вниз по дереву, для каждого узла выставаем значение левого дочернего узла, а правый обновляем с учетом этой суммы.

Так как массив может быть большим в blelloch scan мы разбиваем обработку на несколько этапов. Если массив не превышает размер shared памяти, то мы выполняем в 1 вызов up-down sweep. В этом одном вызове, если нужно, дополняем массив, чтобы стал кратен размеру shared памяти. После просто вызываем up-down sweep. Большие же массивы разбиваем на блоки, если он неполный, дополняем его нулями. После просчета up-down sweep извлекаем последние элементы для последующей корректировки префиксных сумм между блоками. Полученные последние элементы снова прогоняем рекурсивно через blelloch scan для корректировки.

Расстановка чисел для текущего разряда разбита на две части и переиспользует существующую память. С одной стороны плюс, так как мы не выделяем дополнительную память, а с другой код становится полностью нечитаемым. В первой функции мы вычисляем позиции, используя массив битов и префиксных сумм, во второй мы расставляем элементы по своим местам, используя массив битов как хранилище под позиции, а префиксные суммы под хранение элементов.

Описание программы

Программа в одном файле. Читает числа в hex из stdin и записывает в том же формате в stdout. Выделяем память под массивы и запускаем radixSort. В ней выделяем память под префиксные суммы и биты и проходимся поразрядно по всем числам. Получаем биты для текущего разряда при помощи ядра getBits. Для текущего массива битов вызываем функцию blellochScan, которая посчитает префиксную сумму для битов. После вызовем ядро buildPosition, которое, вычислит позиции в зависимости от префиксной суммы битов и самих битов, и ядро applyPosition, которое расставит числа по новым позициям. Функция blellochScan рекурсивная, отвечает за разбиение массива на блоки, чтобы не обрабатывать его целиком. Размер блоков задан 2^{15} . upDownSweep использует shared память, чтобы считать префиксные суммы в параллель. Ядра getBits, buildPosition, и applyPosition запускаются с фиксированными параметрами параллелизации и обрабатывают в цикле только свои элементы.

Результаты

1. nvprof для массива с длиной 1_000_000 с элементами по убыванию.
BLOCK_SIZE, NUM_BLOCKS 256 и SHARED_SIZE 512

```
✓ 2 OK. !nvprof --version
!nvcc -G -g -o lab_5 lab_5.cu
# !compute-sanitizer --leak-check full ./lab_5 < input.bin > sanitizer_log.txt
!nvprof ./lab_5 < input.bin > output.bin
```

```
nvprof: NVIDIA (R) Cuda command line profiler
Copyright (c) 2012 - 2023 NVIDIA Corporation
Release version 12.2.142 (21)
==6860== NVPROF is profiling process 6860, command: ./lab_5
==6860== Profiling application: ./lab_5
==6860== Profiling result:
   Type  Time(%)   Time     Calls   Avg        Min        Max      Name
GPU activities: 71.54%  47.227ms    96  491.95us  97.790us  1.281ms  upDownSweep(int*, int*, int)
              6.24%   4.1173ms   32  128.67us  107.33us  143.33us  buildPosition(int*, int*, int)
              5.98%   3.9454ms   64  61.647us  5.9200us  118.27us  blllelochSum(int*, int*, int)
              4.83%   3.1872ms   32  99.601us  98.526us  100.89us  applyPosition(int*, int*, int)
              3.38%   2.2342ms   32  69.818us  66.142us  70.559us  getBits(int*, int*, int, int)
              2.51%   1.6542ms  288  5.7430us   768ns   20.352us  [CUDA memset]
              2.01%   1.3262ms   32  41.443us  40.895us  41.855us  [CUDA memcpy DtoD]
              1.17%   770.00us    1  770.00us  770.00us  770.00us  [CUDA memcpy HtoD]
              0.97%   640.53us    1  640.53us  640.53us  640.53us  [CUDA memcpy DtoH]
              0.76%   504.83us    64  7.8870us  6.5270us  9.5680us  getLastElements(int*, int*, int, int)
              0.61%   404.25us    64  6.3160us  6.1760us  6.4320us  blllelochSub(int*, int*, int)
API calls:    42.96%  126.22ms   131  963.54us  3.9340us  121.22ms  cudaMalloc
              34.89%  102.52ms   384  266.99us  3.7490us  98.971ms  cudaLaunchKernel
              20.40%  59.929ms   131  457.47us  3.7180us  1.7107ms  cudaFree
              1.02%   2.9857ms   34  87.814us  15.248us  1.2433ms  cudaMemcpy
              0.61%   1.7938ms  288  6.2280us  2.7850us  21.700us  cudaMemcpySet
              0.07%   192.52us   114  1.6880us  262ns   69.612us  cuDeviceGetAttribute
              0.04%   111.86us   384   291ns    149ns   1.0350us  cudaGetLastError
              0.01%   22.794us    3  7.5980us  283ns   21.402us  cuDeviceGetCount
              0.00%   12.463us    1  12.463us  12.463us  12.463us  cuDeviceGetName
              0.00%   7.4050us    1  7.4050us  7.4050us  7.4050us  cuDeviceGetPCIBusId
              0.00%   5.6290us    1  5.6290us  5.6290us  5.6290us  cuDeviceTotalMem
              0.00%   1.0340us    2    517ns   299ns    735ns  cuDeviceGet
              0.00%    431ns    1    431ns   431ns    431ns  cuModuleGetLoadingMode
              0.00%    423ns    1    423ns   423ns    423ns  cuDeviceGetUuid
```

2. nvprof для массива с длиной 1_000_000 с элементами по убыванию.
BLOCK_SIZE, NUM_BLOCKS 16 и SHARED_SIZE 256

```
✓ 2 OK. !nvprof --version
!nvcc -G -g -o lab_5 lab_5.cu
# !compute-sanitizer --leak-check full ./lab_5 < input.bin > sanitizer_log.txt
!nvprof ./lab_5 < input.bin > output.bin
```

```
nvprof: NVIDIA (R) Cuda command line profiler
Copyright (c) 2012 - 2023 NVIDIA Corporation
Release version 12.2.142 (21)
==7404== NVPROF is profiling process 7404, command: ./lab_5
==7404== Profiling application: ./lab_5
==7404== Profiling result:
   Type  Time(%)   Time     Calls   Avg        Min        Max      Name
GPU activities: 43.27%  265.34ms    32  8.2919ms  5.1646ms  15.826ms  buildPosition(int*, int*, int)
              31.00%  190.11ms    32  5.9410ms  4.6457ms  10.100ms  applyPosition(int*, int*, int)
              20.90%  128.18ms    32  4.0056ms  3.0162ms  6.9934ms  getBits(int*, int*, int, int)
              3.57%   21.896ms   96  228.08us  32.511us  1.1557ms  upDownSweep(int*, int*, int)
              0.35%   2.1645ms   64  33.819us  2.3040us  113.85us  blllelochSum(int*, int*, int)
              0.23%   1.4112ms  288  4.8990us  384ns   20.063us  [CUDA memset]
              0.20%   1.2057ms   32  37.679us  36.384us  41.535us  [CUDA memcpy DtoD]
              0.13%   820.75us   64  12.824us  2.4310us  39.807us  getLastElements(int*, int*, int, int)
              0.13%   780.34us   64  12.192us  2.3680us  39.423us  blllelochSub(int*, int*, int)
              0.11%   680.88us    1  680.88us  680.88us  680.88us  [CUDA memcpy HtoD]
              0.11%   647.54us    1  647.54us  647.54us  647.54us  [CUDA memcpy DtoH]
API calls:    77.52%  599.75ms   131  4.5782ms  3.7100us  34.245ms  cudaFree
              11.82%   91.483ms   131  698.35us  2.5590us  88.161ms  cudaMalloc
              8.90%   68.860ms   384  179.32us  3.3480us  66.384ms  cudaLaunchKernel
              1.57%   12.184ms   34  358.34us  13.277us  10.824ms  cudaMemcpy
              0.15%   1.1756ms  288  4.0810us  1.7430us  19.803us  cudaMemcpySet
              0.02%   136.82us   114  1.2000us  144ns   56.116us  cuDeviceGetAttribute
              0.01%   70.396us   384   183ns    137ns   1.2570us  cudaGetLastError
              0.00%   11.858us    1  11.858us  11.858us  11.858us  cuDeviceGetName
              0.00%   5.4620us    1  5.4620us  5.4620us  5.4620us  cuDeviceGetPCIBusId
              0.00%   4.4460us    1  4.4460us  4.4460us  4.4460us  cuDeviceTotalMem
              0.00%   1.3080us    3    436ns   216ns    808ns  cuDeviceGetCount
              0.00%    981ns    2    490ns   207ns    774ns  cuDeviceGet
              0.00%    588ns    1    588ns   588ns    588ns  cuModuleGetLoadingMode
              0.00%    221ns    1    221ns   221ns    221ns  cuDeviceGetUuid
```

3. nvprof для массива с длиной 100_000_000 с элементами по убыванию. BLOCK_SIZE, NUM_BLOCKS 256 и SHARED_SIZE 256

```
!nvprof --version
!nvcc -G -g -o lab_5 lab_5.cu
# !compute-sanitizer --leak-check full ./lab_5 < input.bin > sanitizer_log.txt
!nvprof ./lab_5 < input.bin > output.bin
```

nvprof: NVIDIA (R) Cuda command line profiler
Copyright (c) 2012 - 2023 NVIDIA Corporation
Release version 12.2.142 (21)
==7888== NVPROF is profiling process 7888, command: ./lab_5
==7888== Profiling application: ./lab_5
==7888== Profiling result:

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	55.51%	1.39914s	128	10.931ms	32.447us	108.48ms	upDownSweep(int*, int*, int)
	9.34%	235.31ms	32	7.3534ms	6.5727ms	13.365ms	buildPosition(int*, int*, int)
	9.06%	228.29ms	32	7.1341ms	6.4971ms	9.3549ms	applyPosition(int*, int*, int)
	6.95%	175.28ms	96	1.8259ms	2.2400us	11.151ms	blellochSum(int*, int*, int)
	5.80%	146.07ms	32	4.5648ms	4.3955ms	6.3622ms	getBits(int*, int*, int)
	4.17%	105.22ms	32	3.2881ms	3.2372ms	3.2945ms	[CUDA memcpy DtoD]
	3.36%	84.564ms	1	84.564ms	84.564ms	84.564ms	[CUDA memcpy DtoH]
	3.34%	84.087ms	1	84.087ms	84.087ms	84.087ms	[CUDA memcpy HtoD]
	2.21%	55.585ms	320	173.70us	352ns	1.7346ms	[CUDA memset]
	0.25%	6.3345ms	96	65.984us	2.6240us	196.60us	getLastElements(int*, int*, int, int, int)
	0.02%	629.97us	96	6.5620us	2.5280us	33.599us	blellochSub(int*, int*, int)
API calls:	89.00%	2.33611s	195	11.980ms	3.0090us	116.99ms	cudaFree
	7.09%	186.14ms	34	5.4748ms	14.805us	101.27ms	cudaMemcpy
	3.68%	96.654ms	195	495.66us	3.0610us	88.981ms	cudaMalloc
	0.16%	4.2602ms	512	8.3200us	3.5000us	871.07us	cudaLaunchKernel
	0.06%	1.5319ms	320	4.7870us	1.7560us	45.032us	cudaMemset
	0.01%	135.50us	114	1.1880us	144ns	55.595us	cuDeviceGetAttribute
	0.00%	102.43us	512	200ns	139ns	2.0220us	cudaGetLastError
	0.00%	11.902us	1	11.902us	11.902us	11.902us	cuDeviceGetName
	0.00%	4.8830us	1	4.8830us	4.8830us	4.8830us	cuDeviceGetPCIBusId
	0.00%	4.1600us	1	4.1600us	4.1600us	4.1600us	cuDeviceTotalMem
	0.00%	2.9580us	2	1.4790us	180ns	2.7780us	cuDeviceGet
	0.00%	1.3400us	3	446ns	185ns	859ns	cuDeviceGetCount
	0.00%	418ns	1	418ns	418ns	418ns	cuModuleGetLoadingMode
	0.00%	333ns	1	333ns	333ns	333ns	cuDeviceGetUuid

4. nvprof для массива с длиной 100_000_000 с элементами по убыванию. BLOCK_SIZE, NUM_BLOCKS 256 и SHARED_SIZE 512

```
!nvprof --version
!nvcc -G -g -o lab_5 lab_5.cu
# !compute-sanitizer --leak-check full ./lab_5 < input.bin > sanitizer_log.txt
!nvprof ./lab_5 < input.bin > output.bin
```

nvprof: NVIDIA (R) Cuda command line profiler
Copyright (c) 2012 - 2023 NVIDIA Corporation
Release version 12.2.142 (21)
==8154== NVPROF is profiling process 8154, command: ./lab_5
==8154== Profiling application: ./lab_5
==8154== Profiling result:

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	56.45%	1.53054s	96	15.943ms	35.903us	120.24ms	upDownSweep(int*, int*, int)
	8.66%	234.86ms	32	7.3393ms	6.5832ms	13.347ms	buildPosition(int*, int*, int)
	8.42%	228.42ms	32	7.1382ms	6.5020ms	9.3594ms	applyPosition(int*, int*, int)
	6.47%	175.32ms	64	2.7394ms	9.6640us	11.497ms	blellochSum(int*, int*, int)
	5.33%	144.52ms	32	4.5163ms	4.3855ms	6.3581ms	getBits(int*, int*, int)
	4.08%	110.66ms	288	384.23us	352ns	1.7301ms	[CUDA memset]
	3.88%	105.24ms	32	3.2887ms	3.2345ms	3.2946ms	[CUDA memcpy DtoD]
	3.49%	94.699ms	1	94.699ms	94.699ms	94.699ms	[CUDA memcpy HtoD]
	3.07%	83.327ms	1	83.327ms	83.327ms	83.327ms	[CUDA memcpy DtoH]
	0.13%	3.3997ms	64	53.119us	2.6880us	105.89us	getLastElements(int*, int*, int, int, int)
	0.01%	362.74us	64	5.6670us	2.5280us	20.192us	blellochSub(int*, int*, int)
API calls:	88.66%	2.51774s	131	19.219ms	5.4880us	130.18ms	cudaFree
	6.89%	195.66ms	34	5.7548ms	15.777us	99.972ms	cudaMemcpy
	4.24%	120.48ms	131	919.67us	3.0490us	114.38ms	cudaMalloc
	0.15%	4.1604ms	384	10.834us	3.4010us	966.04us	cudaLaunchKernel
	0.05%	1.4690ms	288	5.1000us	1.6980us	23.000us	cudaMemset
	0.01%	207.07us	114	1.8160us	253ns	91.406us	cuDeviceGetAttribute
	0.00%	118.96us	384	309ns	140ns	27.806us	cudaGetLastError
	0.00%	15.383us	1	15.383us	15.383us	15.383us	cuDeviceGetName
	0.00%	6.9250us	1	6.9250us	6.9250us	6.9250us	cuDeviceGetPCIBusId
	0.00%	5.6120us	1	5.6120us	5.6120us	5.6120us	cuDeviceTotalMem
	0.00%	2.0600us	3	686ns	347ns	1.1430us	cuDeviceGetCount
	0.00%	1.6000us	1	1.6000us	1.6000us	1.6000us	cuModuleGetLoadingMode
	0.00%	1.6000us	2	580ns	410ns	750ns	cuDeviceGet
	0.00%	360ns	1	360ns	360ns	360ns	cuDeviceGetUuid

По результатам nvprof видно, что основное потребление гри - это upDownSweep и cudaFree. Это означает, что использование памяти достаточно оптимальное и мы

упираемся в параллелизацию или эффективность самого алгоритма. Также видно, что 512 - достаточно оптимальное значение для `SHARED_SIZE`, так как мы уменьшили количество вызовов `upDownSweep`, но выполняем его для большего количества элементов и время почти не поменялось. Также видим, что 256 для `BLOCK_SIZE`, `NUM_BLOCKS` оптимально, так как ядра, которые параллелятся от этих значений используют небольшую часть `gru` и выполняются относительно быстро. Во втором примере видно что будет, если подобрать неоптимальные значения для этих переменных.

Выводы

Данный алгоритм можно применять для обработки больших объемов данных с нормальной скоростью. Чаще всего используется в оптимизации графических алгоритмов (упорядочение пикселей или вершин графа), в физическом моделировании или биоинформатике, в базах данных для ускорения индексации данных, а также в машинном обучении для агрегирования признаков. Так как примеров чтения hex чисел не было пришлось самостоятельно попробовать и перебрать несколько вариантов. Также пришлось самостоятельно искать более подробное и понятное объяснение алгоритма `blelloch`.

Была масса проблем, еще до множества посылок) Первая связана с плохо написанной индексацией в `up-down sweep` и самом `blelloch`, из-за чего половина данных затиралась. Вторая с синхронизацией потоков в ядре `up-down sweep` из-за чего на больших данных неправильно сортировал из-за конфликтов. Третья была в том, что я не предусмотрел момент, что размер массива для `up-down sweep` должен быть степенью двойки. Четвертая была в неправильном разбиении на блоки. А потом я отправил программу в чекер и начал возиться с памятью. Убрал кучу ненужных буфферов и напоследок, придумал как оптимально сортировать массив переиспользуя уже выделенные массивы. Каждую ошибку отлавливал, написав тест, который генерирует массив с обратным порядком чисел, запускает программу и сравнивает с прямым порядком чисел. Благодаря этому учитывались все случаи с длинной массива. Также использовал утилиту `compute-sanitizer`, чтобы отлавливать утечки в памяти.